

SIP Profile API Specification

Version 1.0; July 5, 2004

SIP

NOKIA

Copyright © 2004 Nokia Corporation. All rights reserved.

Nokia and Nokia Connecting People are registered trademarks of Nokia Corporation. Java and all Java-based marks are trademarks or registered trademarks of Sun Microsystems, Inc. Other product and company names mentioned herein may be trademarks or trade names of their respective owners.

Disclaimer

The information in this document is provided “as is,” with no warranties whatsoever, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of any proposal, specification, or sample. Furthermore, information provided in this document is preliminary, and may be changed substantially prior to final release. This document is provided for informational purposes only.

Nokia Corporation disclaims all liability, including liability for infringement of any proprietary rights, relating to implementation of information presented in this document. Nokia Corporation does not warrant or represent that such use will not infringe such rights.

Nokia Corporation retains the right to make changes to this specification at any time, without notice.

License

A license is hereby granted to download and print a copy of this specification for personal use only. No other license to any other intellectual property rights is granted herein.

Contents

1	Introduction.....	5
1.1	List of Required Interfaces	5
1.2	Product Platform and/or Symbian OS Version Constraints.....	5
2	Technical Specification.....	6
2.1	Interface Type.....	6
2.2	Interface Class Structure	6
2.3	Usage.....	7
2.3.1	Protocol	7
2.3.2	Error handling	8
2.3.3	Memory overhead.....	9
2.3.4	Extensions to the API.....	9
2.4	Examples	9
2.5	Detailed Description	11
2.5.1	CSIPProfileRegistryBase	11
2.5.2	CSIPProfileRegistry.....	13
2.5.3	CSIPProfile.....	16
2.5.4	MSIPProfileRegistryObserver	22
2.6	Code Architecture	25
3	Index	26
3.1	List of Constants and Enums.....	26
3.2	Alphabetical List of Classes and Functions.....	26
3.3	TSIPProfileTypeInfo	27
4	Terms and Abbreviations	28
5	References	29

Change History

July 5, 2004	Version 1.0	Initial document release

1 Introduction

The Session Initiation Protocol (SIP) is the Internet's de facto application layer signaling protocol for creating, modifying, and terminating session(s) with participant(s). SIP extensions are used for instant messaging, subscribing to events, receiving event notifications, and publishing presence information. SIP is adopted by the 3GPP and used in IP Multimedia System (IMS) Release 5. SIP can run on top of several different transport protocols.

The SIP Profile Agent API provides a SIP registration service for applications. The behavior of the registration service depends on the profile type used. There are different profile types for different standards, for example, 3GPP R5 IP Multimedia System, Internet Engineering Task Force (IETF), and proprietary standard types. At any given time, zero or more profiles of any profile type may exist. Each profile has a one-to-one correspondence with one SIP registration. The API allows more than one application to share the same profile, and thus the same SIP registration.

The user of the API can be unaware of the details of the registration process and can ignore state changes of the access point connection during SIP registration.

The SIP Profile Agent subsystem stores the SIP profiles in the persistent storage. As mentioned before, these profiles hold information necessary for providing required registration behavior. A profile stores the following main parameters:

- The user-friendly name of the service provider.
- The type of the profile, for example, 3GPP R5 IMS, IETF, or a proprietary profile type.
- The user's registered public name (SIP Address Of Record, AOR).
- The access provider identifier of the used Internet Access Point (IAP).
- The address of the SIP registrar server, with optional connection and security parameters.
- The address of the SIP proxy server, with optional connection and security parameters.
- Compression and security preferences.
- An indication of whether the profile is to be always automatically registered or only when it is used by an application.
- Any number of extension parameters stored as key/value pairs.

The SIP profiles can be managed with the SIP Settings UI (by the end user), or they can be managed with over the air (OTA) configuration (by the SIP service provider).

1.1 List of Required Interfaces

In order for an application to use this API, the SIP Client API is needed as well. For further details, see the document *SIP Client API Specification* [1].

1.2 Product Platform and/or Symbian OS Version Constraints

This API is valid for all the platforms running Symbian OS v7.0s.

2 Technical Specification

2.1 Interface Type

The logical type of the Profile Agent API is both Library API and Settings API. The interface allows multiple clients to share common resources, profiles, and SIP registrations. The API also provides access to permanently stored profiles and allows reading the configuration values of profiles.

The technical type of the API is Server Client API.

All the time-consuming operations of the API are asynchronous.

2.2 Interface Class Structure

A client of the SIP Profile API must create an object of type `CSIPProfileRegistry`. In order to do so, the client must have instantiated the `CSIP` class. In addition to that, the client must also implement the callback functions defined in the `MSIPProfileRegistryObserver` interface.

An instance of `CSIPProfile` is created using a retrieve function of the `CSIPProfileRegistry` class. This will read the profile data from the permanent store and copy the data to the created instance. The copied values can be read using the getter functions of `CSIPProfile`.

The client can take the profile into use by enabling it through `CSIPProfileRegistry`. To do this, the client must implement `MSIPConnectionObserver`. Enabling the profile will cause the profile to register itself if it is not yet registered. In this case, the client is notified about the registration status through the `ProfileRegistrationStatusChanged()` function of `MSIPProfileRegistryObserver`. By enabling the profile, the client indicates to the API that the profile is being used by someone, and thus cannot be removed. Even in this case, the profile may be updated, although this is not recommended.

The client can stop using the profile by disabling it. The client can then delete the instance of `CSIPProfile`.

The class structure of the API is shown in Figure 1.

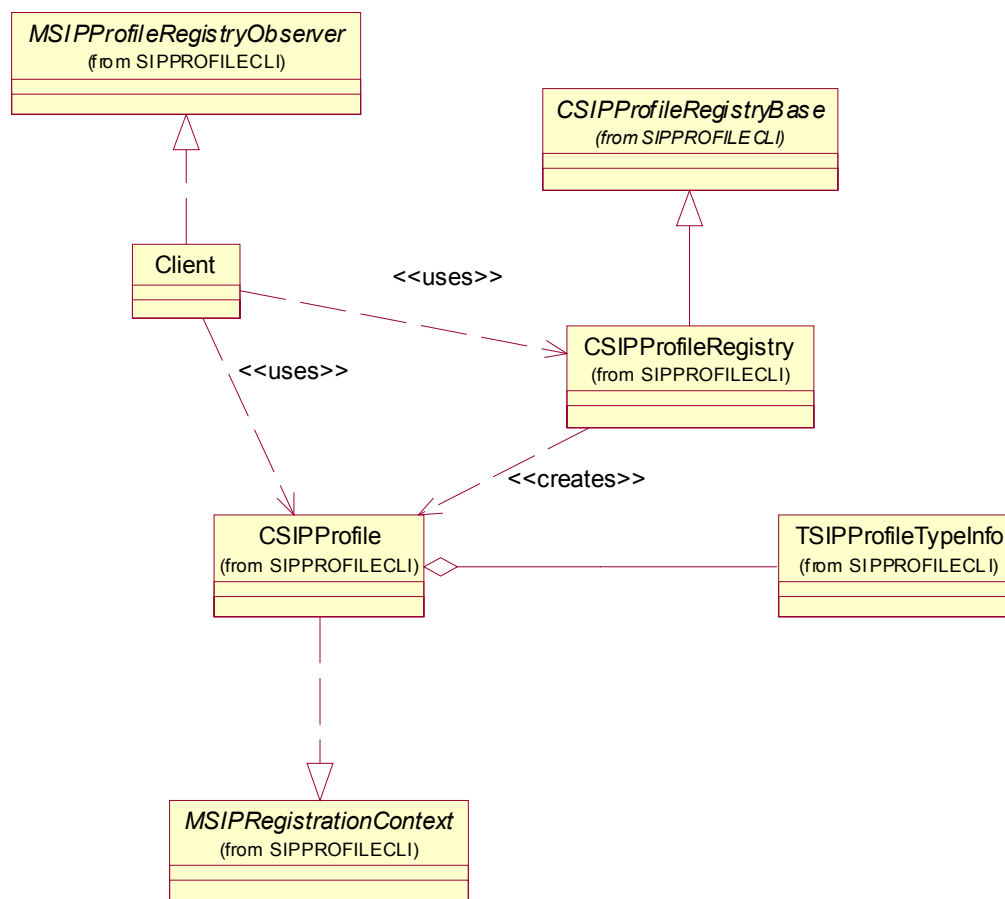


Figure 1: Interface class structure

2.3 Usage

2.3.1 Protocol

Instantiating the class `CSIPProfileRegistry` connects a client to the shared SIP Profile server. If the server is not currently running, it is started by the API implementation. When the `CSIPProfileRegistry` object is deleted, the corresponding connection to the server is closed. The server may be stopped at this point if it has no other users.

Retrieving an instance of `CSIPProfile` through `CSIPProfileRegistry` causes the profile data to be transferred from the server to the client. As long as the client holds this instance of `CSIPProfile` in its memory, all events related to the corresponding profile are notified to that client through functions of `MSIPProfileRegistryObserver` implemented by the client. When the client deletes the instance of `CSIPProfile`, events are no longer sent. Each retrieval of a profile creates a separate instance of class `CSIPProfile`, even if the corresponding profile is the same.

Before an instance of class `CSIPProfile` can be used by the SIP Client API, the profile must be active. This can be verified using the `MSIPRegistrationContext::IsContextActive()` function. The profile becomes active when it is enabled by the client and the state of the profile is `ERegistered`. Enabling the profile leads automatically to registration if the profile was not already registered. In this case the registration status change is notified through `MSIPProfileRegistryObserver`. If the enabled profile was already registered, no additional registration or status change event are sent. The

client of the API must check the return value of `CSIPProfileRegistry::EnableL` to see whether or not the client must wait for the registration to complete before the profile can be used.

After a profile has been enabled and registered, the client can use it to create SIP sessions with the SIP Client API. Enabling the profile also guarantees that it is not removed from the system while being used by the client.

When a profile is no longer used by the client, it should be disabled. Deleting the instance of the profile from the client's memory leads automatically also to disabling that profile.

See Figure 2 for the typical use of the API.

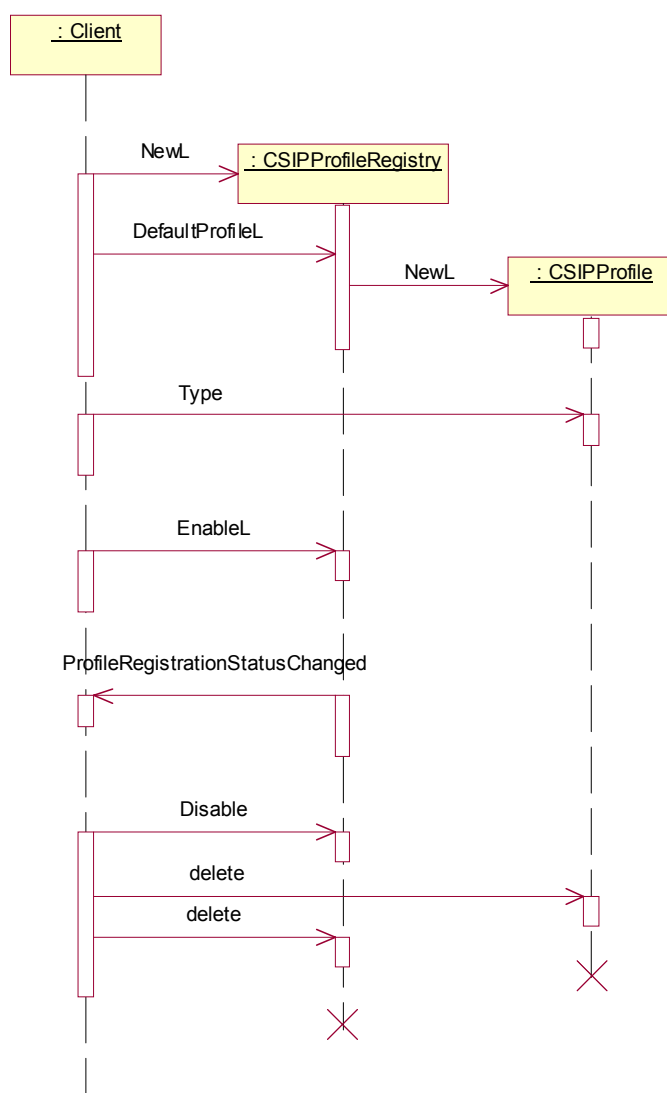


Figure 2: The protocol of the API

2.3.2 Error handling

Errors are handled in this API either by having functions leave and/or by using callback functions.

If an error occurs during a synchronous operation initiated by the client, the called function leaves with a proper error code.

If an error occurs during an asynchronous operation, the error is passed to the client through the callback function `MSIPProfileRegistryObserver::ProfileRegistryErrorOccurred()`. In this case, both the profile identifier and an error code are passed to the client.

Any error passed to the client means that the indicated profile is no longer registered or enabled by the client.

2.3.3 Memory overhead

The amount of overhead memory used by a profile depends on its content. A typical usage is about 0.2 KB per profile. In addition to that, the SIP Profile Server monitors all instantiated profiles in order to send asynchronous events to clients when needed. It is therefore recommended that clients instantiate only the minimum number of profiles they need and delete the instances immediately when they are no longer needed.

2.3.4 Extensions to the API

There are no possible extensions to this API.

2.4 Examples

This section presents code examples describing an implementation for the simple use of the API.

Instantiating CSIPProfileRegistry

```
// An instance of SIP client is needed. For that, client must
// implement MSIPObserver from SIP Client API.
iSip = CSIP::NewL(iDoc.AppUid(), *this);

// For instantiating CSIPProfileRegistry, the client must
// implement MSIPProfileRegistryObserver.
iProfileRegistry = CSIPProfileRegistry::NewL(*iSip, *this);
```

Retrieving profiles by AOR

```
// For retrieving a number of profiles, the client must instantiate
// an RPointerArray for holding instances of CSIPProfile.
RPointerArray<CSIPProfile> profiles;

// Profiles matching a given AOR are returned in an array.
iProfileRegistry->ProfilesL(_L8( "sip:jane.doe@foo.com" ), profiles );
...

// When a client retrieves a profile, its ownership is transferred to
// the client, so it is responsible for deleting the instances when they
// are no longer needed.
profiles.ResetAndDestroy();
```

Retrieving profiles by type

```
// For retrieving a number of profiles, the client must instantiate
// an RPointerArray for holding instances of CSIPProfile.
RPointerArray<CSIPProfile> profiles;

// The desired type is defined by using a variable of type
// TSIPProfileTypeInfo.
TSIPProfileTypeInfo type;
type.iSIPProfileClass = TSIPProfileTypeInfo::EInternet;
```

```

type.iSIPProfileName = _L8("IETF");

// Profiles matching a given AOR are returned in an array
// Profiles matching a given type are returned in an array.
iProfileRegistry->ProfilesL(type, profiles);
...
// When a client retrieves a profile, its ownership is transferred to
// the client, so it is responsible for deleting the instances when they
// are no longer needed.
profiles.ResetAndDestroy();

```

Retrieving a default profile

```

// Retrieving a default profile.
CSIPProfile* profile = iProfileRegistry->DefaultProfileL();

// Ownership of the retrieved profile instance is transferred to the
// client when it retrieves the instance, so the client is responsible
// for deleting it when it is no longer needed.
delete profile;

```

Enabling a profile

```

// Retrieving a default profile. The instance must be stored and must
// not be deleted before the profile is disabled.
iProfile = iProfileRegistry->DefaultProfileL();

// The client asks the API to enable the retrieved profile for its use.
// For that, the client must implement MSIPConnectionObserver.
TInt status = iProfileRegistry->EnableL( *iProfile, *this );

// The returned status indicates if the profile can be immediately used
// for creating a session, or if the client must wait for the profile to
// be registered.
if (status == KErrNone)
{
    // get the SIP connection used by the profile
    iConnection = iProfileRegistry->ConnectionL(*iProfile);
    // create a SIP dialog for sending an INVITE;
    // toHeader contains the SIP address of the invited party
    iDialog = CSIPInviteDialogAssoc::NewL( *iConnection, *toHeader,
    *iProfile );
    //Continue session establishment using the SIP Client API
    ...
}
else
{
    // wait for the profile to be registered before using it further
}

```

Getting registration events

```

// A callback function, which must be implemented by the client, is
// called when the registration status of a profile owned by the client
// is changed.
void CMySIPClient::ProfileRegistrationStatusChanged( TInt32 aProfileId
)
{
    if ( aProfileId == iProfile->Id() )
    {
        // The status of matching profile is checked.
        CSIPProfile::TStatus status = iProfile->Status();
        ...
    }
}

```

Disabling a profile

```
// A client should disable an enabled profile when it no longer needs
// to use it.
TInt status = iProfileRegistry->Disable(*iProfile);

// Finally, the client should delete the profile.
delete iProfile;
```

2.5 Detailed Description

2.5.1 CSIPProfileRegistryBase

CSIPProfileRegistryBase is an abstract class that provides the profile registry functionality that is common to all registries.

2.5.1.1 ProfilesL

Function	<code>void ProfilesL(const TSIPProfileTypeInfo& aType, RPointerArray<CSIPProfile>& aProfiles);</code>
Description	Gets all SIP profiles of a particular type; leaves on failure. The ownerships of the returned profiles are transferred to the caller.
Preconditions	None
Parameters	<code>const TSIPProfileTypeInfo& aType</code> : A SIP profile type that is used for a query. <code>RPointerArray<CSIPProfile>& aProfiles</code> : A reference to a client-owned array, which is filled with profiles of the given type on return.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.2 ProfilesL

Function	<code>void ProfilesL(const TDesC8& aAOR, RPointerArray<CSIPProfile>& aProfiles);</code>
Description	Gets all SIP profiles for a particular SIP AOR; leaves on failure. The ownerships of the returned profiles are transferred to the caller.
Preconditions	None
Parameters	<code>const TDesC8& aAOR</code> : A SIP AOR, for example, user@domain.com. <code>RPointerArray<CSIPProfile>& aProfiles</code> : A reference to a client-owned array, which is filled with profiles of the given AOR on return.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.3 ProfileL

Function	<code>CSIPProfile* ProfileL(TUint32 aId);</code>
Description	Gets the SIP profile identified by <code>aId</code> ; leaves on failure. The ownership of the returned profile is transferred to the caller.
Preconditions	None
Parameters	<code>TUint32 aId: aId</code> , a SIP profile ID.
Return value	<code>CSIPProfile*</code> : A SIP profile or NULL if such a profile does not exist.
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with <code>KErrArgument</code> if <code>aId <= 1</code> .

2.5.1.4 DefaultProfileL

Function	<code>CSIPProfile* DefaultProfileL();</code>
Description	Gets the default SIP profile; leaves on failure. The ownership of the returned profile is transferred to the caller.
Preconditions	None
Parameters	None
Return value	<code>CSIPProfile*</code> : A SIP profile or NULL if such a does not exist.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.5 SupportedProfileTypesL

Function	<code>void SupportedProfileTypesL(RArray<TSIPProfileTypeInfo>& aSupportedProfileTypes);</code>
Description	Gets the supported profile types; leaves on failure. The ownerships of the returned objects are transferred to the caller.
Preconditions	None
Parameters	<code>RArray<TSIPProfileTypeInfo>& aSupportedProfileTypes</code> : A reference to a client-owned array, which is filled with supported profile types on return. Any existing data in the array is destroyed.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.6 LastRegistrationError

Function	<code>TInt LastRegistrationError(const CSIPProfile& aProfile) const;</code>
Description	Gets the last error that occurred during the registration of the given profile. The error codes are either local, defined by Symbian OS and SIP Client API, or they originate from the network and are defined by the SIP protocol.
Preconditions	None
Parameters	<code>const CSIPProfile& aProfile</code> : A profile to be checked.
Return value	<code>TInt</code> : The last registration error that has occurred. <code>KErrNone</code> if no errors have occurred.
Postconditions	None
Exceptions	None

2.5.2 CSIPProfileRegistry

`CSIPProfileRegistry` is a class that provides the profile registry functionality for SIP clients that can only read SIP profile data.

2.5.2.1 NewL

Function	<code>static CSIPProfileRegistry* NewL(CSIP& aSip, MSIPProfileRegistryObserver& aObserver);</code>
Description	Creates a new instance of <code>CSIPProfileRegistry</code> . This constructor should be used if the client intends to use SIP services with profiles.
Preconditions	None
Parameters	<code>CSIP& aSip</code> : A SIP client providing the connection to the SIP server. <code>MSIPProfileRegistryObserver& aObserver</code> : An observer for the SIP profile registry's change events.
Return value	<code>CSIPProfileRegistry*</code> : A new instance of <code>CSIPProfileRegistry</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.2.2 *NewLC*

Function	<code>static CSIPProfileRegistry* NewLC(CSIP& aSip, MSIPProfileRegistryObserver& aObserver);</code>
Description	Creates a new instance of <code>CSIPProfileRegistry</code> and puts it onto the cleanup stack. This constructor should be used if the client intends to use SIP services with profiles.
Preconditions	None
Parameters	<code>CSIP& aSip</code> : A SIP client providing a connection to the SIP server. <code>MSIPProfileRegistryObserver& aObserver</code> : An observer for the SIP profile registry's change events.
Return value	<code>CSIPProfileRegistry*</code> : A new instance of <code>CSIPProfileRegistry</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.2.3 *~CSIPProfileRegistry*

Function	<code>~CSIPProfileRegistry();</code>
Description	Closes the connection to the server and deletes the resources of <code>CSIPProfileRegistry</code> .
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.2.4 *SIP*

Function	<code>CSIP& SIP() const;</code>
Description	Gets a handle to the SIP server.
Preconditions	None
Parameters	None
Return value	<code>CSIP&</code> : A handle to the SIP server.
Postconditions	None
Exceptions	None

2.5.2.5 *ConnectionL*

Function	<code>CSIPConnection* ConnectionL(CSIPProfile& aSIPProfile);</code>
Description	Gets the SIP connection to be used with a given SIP profile. The ownership of the returned connection is transferred to the caller. Note that the returned instance is not always new. If a connection for the same IAP has already been instantiated, the instance corresponding to that connection is returned here. It is the responsibility of the client to delete instances correctly.
Preconditions	<code>IsEnabled() == ETrue</code>
Parameters	<code>CSIPProfile& aSIPProfile</code> : A SIP profile.
Return value	<code>CSIPConnection*</code> : An instance of a SIP connection.
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with <code>KErrArgument</code> if <code>MSIPConnectionObserver</code> is not set.

2.5.2.6 *EnableL*

Function	<code>TInt EnableL(CSIPProfile& aSIPProfile, MSIPConnectionObserver &aObserver);</code>
Description	Enables the SIP profile for use; leaves on failure. Enabling the SIP profile will cause it to be registered, if its status was unregistered. The client is unable to use a SIP registration based on that profile with the SIP Client API unless it has been enabled and registered. The function will leave with <code>KErrNotFound</code> if a nonexistent profile is given as the first argument.
Preconditions	None
Parameters	<code>CSIPProfile& aSIPProfile</code> : A SIP profile to be enabled. <code>MSIPConnectionObserver& aObserver</code> : An observer for SIP connection events.
Return value	<code>TInt</code> : <code>KErrNone</code> if the SIP profile was successfully enabled; <code>KRequestPending</code> if the registration of the SIP profile is required. In this case, the user of the API must wait until the registration state of the profile changes. The SIP profile will be enabled for use when the registration status of the SIP profile is changed to <code>CSIPProfile::TStatus::ERegistered</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with <code>KErrArgument</code> if <code>aSIPProfile.IsEnabled()</code> returns <code>ETrue</code> and with <code>KErrNotFound</code> if a nonexistent profile is given as the first argument.

2.5.2.7 Disable

Function	<code>TInt Disable(CSIPProfile& aSIPProfile);</code>
Description	Disables the usage of the SIP profile.
Preconditions	None
Parameters	<code>CSIPProfile& aSIPProfile</code> : The SIP profile to be disabled.
Return value	<code>TInt</code> : <code>KerrNone</code> if the SIP profile was successfully disabled.
Postconditions	None
Exceptions	None

2.5.2.8 IsEnabled

Function	<code>TBool IsEnabled(const CSIPProfile& aSIPProfile) const;</code>
Description	Checks if the SIP profile is enabled for the use of the client.
Preconditions	None
Parameters	<code>const CSIPProfile& aSIPProfile</code> : The SIP profile to be checked.
Return value	<code>TBool</code> : <code>ETrue</code> if the SIP profile is enabled; otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.3 CSIPProfile

`CSIPProfile` is a class that provides an interface for reading profile data. The client cannot instantiate this class.

2.5.3.1 ~CSIPProfile

Function	<code>~CSIPProfile();</code>
Description	Deletes the resources of <code>CSIPProfile</code> . If the profile was enabled by the client, it is automatically disabled.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.3.2 *Id*

Function	<code>TUint32 Id() const;</code>
Description	Gets the unique profile ID.
Preconditions	None
Parameters	None
Return value	<code>TUint32</code> : ID of the profile.
Postconditions	None
Exceptions	None

2.5.3.3 *Type*

Function	<code>const TSIPProfileTypeInfo& Type() const;</code>
Description	Gets profile type information.
Preconditions	None
Parameters	None
Return value	<code>const TSIPProfileTypeInfo&</code> : Type of the profile.
Postconditions	None
Exceptions	None

2.5.3.4 *IapId*

Function	<code>TUint32 IapId() const;</code>
Description	Gets the access point (IAP) ID of the profile.
Preconditions	None
Parameters	None
Return value	<code>TUint32</code> : IAP-id used for this profile.
Postconditions	None
Exceptions	None

2.5.3.5 Server

Function	<code>const TDesC8& Server(TServer aServer) const;</code>
Description	Gets the URI of the server associated with this profile.
Preconditions	None
Parameters	<code>TServer aServer</code> : A server to get; either <code>EOutboundProxy</code> or <code>ERegistrar</code> .
Return value	<code>const TDesC8&</code> : The URI of the server of the given type associated with this profile. A zero-length descriptor if the server is not defined.
Postconditions	None
Exceptions	None

2.5.3.6 ServerParameter

Function	<code>const TDesC8& ServerParameter(TServer aServer, TServerParameter aServerParam) const;</code>
Description	Gets a particular server parameter.
Preconditions	None
Parameters	<code>TServer aServer</code> : A server to get; either <code>EOutboundProxy</code> or <code>ERegistrar</code> . <code>TServerParameter aServerParam</code> : A parameter to get; either <code>EDigestRealm</code> or <code>EDigestUsername</code> .
Return value	<code>TDesC8&</code> : The parameter of a given type of server of this profile. A zero-length descriptor if the parameter is not defined.
Postconditions	None
Exceptions	None

2.5.3.7 *ServerExtensionParameter*

Function	<code>const TDesC8& ServerExtensionParameter(TServer aServer, const TDesC8& aName) const;</code>
Description	Gets the server extension parameter.
Preconditions	None
Parameters	<code>TServer aServer</code> : The server to get; either <code>EOutboundProxy</code> or <code>ERegistrar</code> . <code>const TDesC8& aName</code> : The name of the parameter to get.
Return value	<code>TDesC8&</code> : The parameter of the given name, associated with the server of the given type, of this profile. A zero-length descriptor if the parameter is not defined.
Postconditions	None
Exceptions	None

2.5.3.8 *ExtensionParameter*

Function	<code>const TDesC8& ExtensionParameter(const TDesC8& aName) const;</code>
Description	Gets the extension parameter of the profile.
Preconditions	None
Parameters	<code>const TDesC8& aName</code> : The name of the parameter to get.
Return value	<code>TDesC8&</code> : The extension parameter of this profile. A zero-length descriptor if the parameter is not defined.
Postconditions	None
Exceptions	None

2.5.3.9 *AORs*

Function	<code>const MDesC8Array& AORs() const;</code>
Description	Gets all the AORs defined for the profile.
Preconditions	None
Parameters	None
Return value	<code>const MDesC8Array&</code> : All AORs of the profile.
Postconditions	None
Exceptions	None

2.5.3.10 Name

Function	<code>const TDesC8& Name() const;</code>
Description	Gets the user-friendly name of the service provider.
Preconditions	None
Parameters	None
Return value	<code>const TDesC8&</code> : The name of the profile. A zero-length descriptor if the service provider is not defined.
Postconditions	None
Exceptions	None

2.5.3.11 IsDefaultL

Function	<code>TBool IsDefaultL() const;</code>
Description	Checks if the profile is the default profile. Only one profile can be the default profile at a time.
Preconditions	None
Parameters	None
Return value	<code>TBool</code> : <code>ETrue</code> if the profile is the default profile, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.12 IsSigCompEnabled

Function	<code>TBool IsSigCompEnabled() const;</code>
Description	Checks if signaling compression (SigComp) is enabled.
Preconditions	None
Parameters	None
Return value	<code>TBool</code> : <code>ETrue</code> if signaling compression is enabled, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.3.13 IsSecurityNegotiationEnabled

Function	<code>TBool IsSecurityNegotiationEnabled() const;</code>
Description	Checks if the profile uses SIP security negotiation.
Preconditions	None
Parameters	None
Return value	<code>TBool</code> : <code>ETrue</code> if the profile uses security negotiation, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.3.14 IsAutoRegistrationEnabled

Function	<code>TBool IsAutoRegistrationEnabled() const;</code>
Description	Checks if the profile is marked for autoregistration.
Preconditions	None
Parameters	None
Return value	<code>TBool</code> : <code>ETrue</code> if autoregistration is set, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.3.15 Status

Function	<code>TStatus Status() const;</code>
Description	Gets the registration status of the profile.
Preconditions	None
Parameters	None
Return value	<code>TStatus</code> : The registration status, either <code>ERegistered</code> or <code>EUnregistered</code> .
Postconditions	None
Exceptions	None

2.5.3.16 *IsContextActive*

Function	<code>TBool IsContextActive() const;</code>
Description	Checks if the registration context can be used for creating SIP messages/dialogs. The profile can be used when <code>CSIPProfileRegistry::IsEnabled() == ETrue</code> and <code>Status() == ERegistered</code> .
Preconditions	None
Parameter	None
Return value	<code>TBool: ETrue</code> if the registration binding exists; otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.3.17 *ContextId*

Function	<code>TUint32 ContextId() const;</code>
Description	Gets the context ID. This function is for internal use only.
Preconditions	None
Parameter	None
Return value	<code>TUint32: Context ID</code> .
Postconditions	None
Exceptions	None

2.5.4 MSIPProfileRegistryObserver

`CSIPProfileRegistryObserver` is a class that a client must implement in order to receive asynchronous events from the SIP Profile API implementation.

2.5.4.1 ProfileCreated

Function	<code>virtual void ProfileCreated(TUint32 aSIPProfileId) = 0</code>
Description	Signals the creation of a SIP profile, indicating that a new profile is added to the permanent storage. This event is sent to all observers.
Preconditions	None
Parameters	TUint32 aSIPProfileId: The ID of the new SIP profile.
Return value	void
Postconditions	None
Exceptions	None

2.5.4.2 ProfileUpdated

Function	<code>virtual void ProfileUpdated(TUint32 aSIPProfileId) = 0;</code>
Description	Signals the updating of a SIP profile, indicating that the data of an existing profile is updated to the permanent storage. This event is sent to those observers who have the corresponding profile instantiated. Newly updated values of the profile are available for clients after they receive this signal. If the profile was registered while updated, reregistration of the profile may occur. Updated values of the profile are uploaded to the network when the SIP registration process is completed. It is possible (but not recommended) that the profile is updated even when it is enabled by the client. If the client receives this event for an enabled profile, the client should react as if the error occurred with the exception indicating that the profile was disabled or deleted first.
Preconditions	None
Parameters	TUint32 aSIPProfileId: The ID of the updated SIP Profile.
Return value	void
Postconditions	None
Exceptions	None

2.5.4.3 ProfileRegistrationStatusChanged

Function	<code>virtual void ProfileRegistrationStatusChanged(TUint32 aSIPProfileId) = 0;</code>
Description	Indicates that the registration status of a SIP profile has been changed. This event is sent to those observers who have the corresponding profile instantiated.
Preconditions	None
Parameters	TUint32 aSIPProfileId: The ID of the profile whose registration status has been changed.
Return value	void
Postconditions	None
Exceptions	None

2.5.4.4 ProfileDestroyed

Function	<code>virtual void ProfileDestroyed(TUint32 aSIPProfileId) = 0;</code>
Description	Signals the destruction of a SIP profile, indicating that the profile has been removed from the permanent store and cannot be used anymore. This event is sent to those observers who have the corresponding profile instantiated.
Preconditions	None
Parameters	TUint32 aSIPProfileId: The ID of the profile which was destroyed.
Return value	void
Postconditions	None
Exceptions	None

2.5.4.5 ProfileRegistryErrorOccurred

Function	<code>virtual void ProfileRegistryErrorOccurred(TUint32 aSIPProfileId, TInt aError) = 0;</code>
Description	Signals the occurrence of an asynchronous error related to a SIP profile. This event is sent to those observers who have the corresponding profile instantiated.
Preconditions	None
Parameters	<code>TUint32 aSIPProfileId</code> : The ID of the profile associated with the failure. <code>TInt aError</code> : An error code.
Return value	<code>void</code>
Postconditions	None
Exceptions	None

2.6 Code Architecture

All services offered by the SIP Profile API are provided as a dynamically linked library called `sipprofilecli.dll`. The header files of the API are located in `/epoc32/include` and are the following:

Header file	Explanation
<code>sipprofile.h</code>	<code>class CSIPProfile</code>
<code>sipprofileregistry.h</code>	<code>class CSIPProfileRegistry</code>
<code>sipprofileregistrybase.h</code>	<code>class CSIPProfileRegistryBase</code>
<code>sipprofileregistryobserver.h</code>	<code>class MSIPProfileRegistryObserver</code>
<code>sipprofiletypeinfo.h</code>	<code>class TSIPProfileTypeInfo</code>
<code>sipconcreteprofileobserver.h</code>	For internal use.

3 Index

3.1 List of Constants and Enums

Class CSIPProfile

```
enum TServer
{
    /** outbound proxy server */
    EOutboundProxy,
    /** registrar server */
    ERegistrar
};

/** Parameters for SIP server */
enum TServerParameter
{
    /** Server parameter realm */
    EDigestRealm,
    /** Server parameter username */
    EDigestUsername
};

/** Profile registration status */
enum TStatus
{
    /** Profile is registered */
    ERegistered,
    /** Profile is unregistered */
    EUnregistered
};
```

Class TSIPProfileTypeInfo

```
/** Profile classes */
enum TSIPProfileClass
{
    /** Network compliant with IETF standards */
    EInternet,
    /** Network compliant with 3GPP standards */
    EImms,
    /** Proprietary */
    EProprietary
};
```

3.2 Alphabetical List of Classes and Functions

Class CSIPProfile

```
IMPORT_C const MDesC8Array& AORs() const;
IMPORT_C ~CSIPProfile();
IMPORT_C const TDesC8& ExtensionParameter(const TDesC8& aName) const;
IMPORT_C TUint32 IapId() const;
IMPORT_C TUint32 Id() const;
IMPORT_C TBool IsAutoRegistrationEnabled() const;
IMPORT_C TBool IsDefaultL() const;
IMPORT_C TBool IsSecurityNegotiationEnabled() const;
IMPORT_C TBool IsSigCompEnabled() const;
IMPORT_C const TDesC8& Name() const;
IMPORT_C const TDesC8& Server(TServer aServer) const;
IMPORT_C const TDesC8& ServerExtensionParameter(
```

```

        TServer aServer,
        const TDesC8& aName) const;
IMPORT_C const TDesC8& ServerParameter(
        TServer aServer,
        TServerParameter aServerParam) const;
IMPORT_C TStatus Status() const;
IMPORT_C const TSIPProfileTypeInfo& Type() const;

```

Class CSIPProfileRegistry

```

IMPORT_C CSIPConnection* ConnectionL(CSIPProfile& aSIPProfile);
IMPORT_C ~CSIPProfileRegistry();
IMPORT_C TInt Disable(CSIPProfile& aSIPProfile);
IMPORT_C TInt EnableL(
        CSIPProfile& aSIPProfile,
        MSIPConnectionObserver &aObserver);
IMPORT_C TBool IsEnabled(const CSIPProfile& aSIPProfile) const;
IMPORT_C static CSIPProfileRegistry* NewL(
        CSIP& aSip,
        MSIPProfileRegistryObserver&
        aObserver);
IMPORT_C static CSIPProfileRegistry* NewLC(
        CSIP& aSip,
        MSIPProfileRegistryObserver&
        aObserver);
IMPORT_C CSIP& SIP() const;

```

Class CSIPProfileRegistryBase

```

IMPORT_C CSIPProfile* DefaultProfileL();
IMPORT_C TInt LastRegistrationError(const CSIPProfile& aProfile) const;
IMPORT_C CSIPProfile* ProfileL(TUint32 aId);
IMPORT_C void ProfilesL(
        const TSIPProfileTypeInfo& aType,
        RPointerArray<CSIPProfile>& aProfiles);
IMPORT_C void ProfilesL(
        const TDesC8& aAOR,
        RPointerArray<CSIPProfile>& aProfiles);
IMPORT_C void SupportedProfileTypesL(
        RArray<TSIPProfileTypeInfo>& aSupportedProfileTypes);

```

Class MSIPProfileRegistryObserver

```

virtual void ProfileCreated(TUint32 aSIPProfileId)=0;
virtual void ProfileDestroyed(TUint32 aSIPProfileId)=0;
virtual void ProfileRegistrationStatusChanged(TUint32 aSIPProfileId)=0;
virtual void ProfileRegistryErrorOccurred(TUint32 aSIPProfileId, TInt
aError)=0;
virtual void ProfileUpdated(TUint32 aSIPProfileId)=0;

```

Class TSIPProfileTypeInfo

3.3 TSIPProfileTypeInfo

```

class TSIPProfileTypeInfo
{
public: // data members
    /** Profile class */
    TSIPProfileClass iSIPProfileClass;
    /** Profile type name */
    TSIPProfileName iSIPProfileName;
};

```

4 Terms and Abbreviations

Term or abbreviation	Meaning
3GPP	3rd Generation Partnership Project
active profile	A profile that is enabled by the client and in registered state.
AOR	Address Of Record, also known as SIP address.
API	Application Programming Interface
default SIP profile	A SIP profile that is configured as default. Only one profile can be default at a given time.
deregistering a profile	Process where users remove their current location information from the network.
disabling a profile	Process where clients give up the use of a SIP profile and SIP registration based on that profile.
enabling a profile	Process where clients take a SIP profile and SIP registration based on that profile into use.
IAP	Internet Access Point
IETF	Internet Engineering Task Force
IMS	IP Multimedia Subsystem
OTA	over the air
profile	A collection of data necessary for providing required SIP registration behavior.
profile instance	An instance of class <code>CSIPProfile</code> that allows the client to access profile data. A client may have more than one instance of class <code>CSIPProfile</code> pointing at the same profile.
profile type	Type of the SIP profile that defines the SIP registration behaviour.
registering a profile	SIP registration based on data stored on SIP profile.
SDK	Software Development Kit
SIP	Session Initiation Protocol
SIP Profile Server	Symbian server providing SIP registration services based on profiles.
SIP registration	Process where users upload their current location information to the network.
SIP Server	Symbian server providing SIP protocol functionality.
SIP settings UI	Application for end the user to add/update/remove SIP profiles.

5 References

SIP Client API Specification. Available at <http://www.forum.nokia.com/sip>.